

4.1 Sincronizar schema — análise do bloco

Contexto: tarefa ClickUp "4.1 Sincronizar schema: conferir migrations aplicadas em dev e prod" (s4/bloco). Escrito a partir da leitura do repositório na branch `feature-dev-breno-automacao` (e2dcc57). **Nada foi executado contra gc-dev ou gc-prod** — o repositório não tem credenciais (`.env.local` não versionado) nem link de CLI.

Atualização 2026-09-02 — gc-prod está pausado. O trabalho acontece só em gc-dev. Isso não invalida nada aqui, mas reordena: conferir e aplicar em prod sai do caminho crítico e vira pendência registrada, e o risco da `004` em prod (destrutiva, 13 colunas de `obras`) passa a ser risco **diferido** — precisa estar em pé no dia em que prod voltar, com backup e janela.

Estado verificado em gc-dev (2026-09-02)

O `verificar_schema_aplicado.sql` foi rodado em gc-dev: **21 de 21 checagens OK**. As migrations **004 a 010 estão todas aplicadas em dev**, inclusive a 010 (bucket `documentos-processamento` privado, 20 MB, `application/pdf` , com as 4 policies de `storage.objects`).

MIGRATION	RESULTADO EM GC-DEV
004	8/8 — <code>clientes</code> , <code>cliente_id</code> NOT NULL, FK, colunas legacy zeradas em <code>obras</code> e <code>orcamentos</code> , rename <code>observacao</code> , 4 views
005	2/2 — RLS + 4 policies em <code>clientes</code>
006	1/1 — <code>calcular_valores_obra</code> com o CASO 2 (<code>fonte=sem_proposta</code>)
007	1/1 — <code>progresso_itens_pct</code> exposto na view
008	4/4 — tabela, RLS, 4 policies, 3 índices
009	3/3 — tabela, RLS, 4 policies
010	2/2 — bucket + 4 policies de storage

Contexto: **16 tabelas e 53 policies** em `public` . As 16 batem com as 14 tabelas que o `types.ts` já conhece mais `documentos_processamento` e `contatos_whatsapp` — coerente com um dev completo e um `types.ts` atrasado.

Types regenerados (2026-09-02). `npm run db:types` rodou contra `gzbmghgnpoehormnidmgg` e trouxe os três objetos que faltavam — o que também prova que **esse project-id é o gc-dev**. O `types.ts` passou de 2267 para 2449 linhas (+192/-10): entraram `contatos_whatsapp` (8 colunas), `documentos_processamento` (13 colunas) e `progresso_itens_pct` em `obras_com_valores` ; as 10 remoções são só reformatação do gerador nos helpers genéricos (`TableName extends ...`). `npx tsc --noEmit` e `npm run lint` seguem limpos.

Consequência prática: dos cinco problemas do diagnóstico, escopo, aplicação e drift de types estão resolvidos em dev — por verificação, não por suposição. Restam os dois estruturais: o cast que esconde drift futuro e a ausência de controle de migrations. Tudo relativo a prod fica em espera.

Resumo (diagnóstico original)

A tarefa, como está escrita, não é executável e os seus critérios de aceite não medem o que deveriam. Cinco problemas, do mais grave pro menos:

#	PROBLEMA	IMPACTO
1	Escopo desatualizado: existem migrations 004 a 010 , não 004 a 007	3 migrations ficariam fora da conferência
2	Não existe controle de migrations (sem <code>supabase/migrations</code> , sem CLI)	"rodar as migrations pendentes" não é um comando; ninguém sabe o que já rodou
3	<code>004</code> é destrutiva e não idempotente	rodar em prod sem janela/backup dropa 13 colunas de <code>obras</code> com dados
4	<code>types.ts</code> está desatualizado	drift silencioso entre schema e código
5	<code>tsc --noEmit</code> e <code>lint</code> passam hoje , com os types velhos	os itens 4 e 5 do checklist são falso-positivos

1. O escopo é 004–010, não 004–007

```
supabase/004_revisao_schema.sql
supabase/005_rls_clientes.sql
supabase/006_fix_calcular_valores_obra.sql
supabase/007_progresso_itens_obra.sql
supabase/008_documentos_processamento.sql      <- commit e2dcc57
supabase/009_contatos_whatsapp.sql              <- commit e2dcc57
supabase/010_storage_bucket_documentos_processamento.sql  <- commit e2dcc57
```

Os caminhos abaixo são os de antes da adoção da CLI. O de-para pra `supabase/migrations/<timestamp>_nome.sql` está em `supabase/README.md`.

008/009/010 entraram no último commit (`migration` e `documentacao`), depois da tarefa ter sido criada. 010 mexe em `storage.buckets` + polices de `storage.objects` — e o workflow n8n de ingestão via WhatsApp depende dele (o próprio arquivo documenta o contrato de path `{empresa_id}/whatsapp/...`). A verificação confirmou a 010 aplicada em **dev**, então a ingestão está de pé no ambiente em uso. Em prod ela é pendência registrada — e do tipo que nenhum `tsc` detecta, porque storage não aparece nos types.

2. Não há como saber o que está aplicado — só introspecção

Não existe `supabase/migrations/`, `supabase/config.toml`, nem projeto linkado pela CLI. Os `.sql` são scripts numerados aplicados à **mão** no SQL Editor. Consequências:

- não existe tabela de controle (`schema_migrations`) pra consultar;
- "rodar as migrations pendentes em gc-dev" não corresponde a nenhum comando;
- a única forma de conferir é **inspecionar o catálogo do Postgres**.

Foi escrito pra isso: `supabase/verificar_schema_aplicado.sql` — somente leitura, seguro em prod. Com prod pausado, roda em dev agora e guarda a saída; prod entra na comparação quando voltar:

```
SQL Editor do gc-dev -> colar -> Run      (agora)
SQL Editor do gc-prod -> colar -> Run      (quando prod despausar)
```

Cada linha sai como OK / FALTA, por migration: tabela `clientes`, colunas legacy de `obras` / `orcamentos` dropadas, FK `obras_cliente_fk`, 4 views recriadas, RLS + policies de `clientes`, o CASO 2 da `calcular_valores_obra` (006), a coluna `progresso_itens_pct` na view (007), `documentos_processamento` (008), `contatos_whatsapp` (009), bucket + policies de storage (010), e contagens totais de tabelas/policies pra diff dev↔prod.

3. Ordem e idempotência: onde isso explode

MIGRATION	IDEMPOTENTE?	EM TRANSAÇÃO?	OBSERVAÇÃO
004	não	sim (<code>begin</code> / <code>commit</code>)	<code>create table clientes</code> e <code>drop column</code> sem <code>if [not] exists</code> ; migra dados de <code>obras</code> para <code>clientes</code>
005	sim	não	<code>drop policy if exists</code> antes de cada <code>create policy</code>
006	sim	n/a	<code>create or replace function</code>
007	sim	n/a	<code>create or replace view</code>
008	não	não	<code>create table documentos_processamento</code> sem guard
009	não	não	<code>create table contatos_whatsapp</code> sem guard
010	sim	não	<code>on conflict do nothing</code> + <code>drop policy if exists</code>

Três armadilhas concretas:

1. 004 depois de 007 reverte o 007. A 004 recria `obras_com_valores` no final (linha 536) sem `progresso_itens_pct`. A listagem de obras seleciona essa coluna explicitamente (`src/app/(app)/obras/page.tsx:51`) — a página quebra em runtime. Ordem obrigatória: 004 -> 005 -> 006 -> 007 -> 008 -> 009 -> 010.

2. **004 sem 006 quebra a listagem inteira.** É o que o cabeçalho do 006 diz: a 004 dropa os campos `legacy` que a `calcular_valores_obra` assumia, e a view chama a função via `LATERAL`. Aplicar 004 em prod e parar ali derruba `/obras`.
3. **008 e 009 não estão em transação.** Se falharem no meio (ex.: index ou policy), deixam estado parcial. Vale envolver em `begin; ... commit;` antes de aplicar em prod.

Sobre a 004 em prod: ela dropa 13 colunas de `obras` (`cliente`, `contato`, `cpf_cnpj`, `telefone`, `email`, `valor_total`, `desconto`, `valor_final`, `pct_*`, `forma_pagamento`, `doc_url`) e 6 de `orcamentos`, depois de copiar os dados pra `clientes`. É irreversível sem backup. O próprio arquivo pede janela de manutenção — o checklist da tarefa ("Aplicar as mesmas migrations em gc-prod") não menciona backup nem janela.

4. `types.ts` está desatualizado

`src/lib/supabase/types.ts` (2267 linhas) **não contém:**

- `progresso_itens_pct` em `obras_com_valores` (007)
- a tabela `documentos_processamento` (008)
- a tabela `contatos_whatsapp` (009)

Ou seja: foi gerado antes do commit `e2dcc57` e antes do 007 chegar na view do projeto de onde foi gerado.

Além disso, `package.json` fixa o projeto no script:

```
"db:types": "npx --yes supabase gen types typescript --project-id gzbmhgnpoehormnidmzg > src/lib/supabase/types.ts"
```

O `--project-id` está hardcoded e o repositório não documenta se `gzbmhgnpoehormnidmzg` é gc-dev ou gc-prod. Não existe forma de gerar types do outro ambiente. Sugestão: `db:types:dev` / `db:types:prod`, ou `--project-id $SUPABASE_PROJECT_ID`.

Cuidado, e já mordeu: o script redireciona com `>` direto pro arquivo final, e o shell trunca o destino *antes* de rodar o comando. Na primeira tentativa a CLI abortou por falta de token (`supabase login`) e o `src/lib/supabase/types.ts` ficou com **0 bytes** — foi restaurado do commit `e2dcc57` (blob `ab59e07`, 2267 linhas). O script agora gera num `.tmp` e só promove com `mv` se a geração der certo, então uma falha deixa o arquivo bom intacto.

Com dev verificado, rodar `npm run db:types` agora responde essa pergunta de graça: se os três objetos ausentes aparecerem no diff, o id é gc-dev e o atraso era só falta de regenerar. Se não aparecerem — ou o comando falhar por projeto indisponível — o id aponta pra gc-prod, que está pausado, e o script precisa mudar antes de qualquer outra coisa.

5. `tsc` e `lint` já passam — e não provam nada

Rodados agora, nesta branch, com os types velhos:

```
npx tsc --noEmit -> sem erros
npm run lint -> No ESLint warnings or errors
```

Os dois itens do checklist já estão verdes **apesar** do drift. Por quê:

- `src/lib/types.ts:232` declara `progresso_itens_pct` à mão;
- `src/app/(app)/obras/page.tsx:96` faz `(data ?? []) as unknown as ObraListItem[]`.

O `as unknown as` apaga qualquer divergência entre o `select(...)` e o `Database` gerado. Enquanto esse cast existir, `tsc` nunca vai detectar coluna faltando — nem agora, nem numa próxima migration. O critério de aceite "npx tsc --noEmit passando sem erro" é, aqui, um teste que não pode falhar.

Checklist — estado em 2026-09-02

Feito

1. ☒ Rodar `supabase/verificar_schema_aplicado.sql` em **gc-dev** — 21/21 OK
2. ☒ Confirmar que 004–010 estão aplicadas em dev, por evidência de catálogo

Feito em dev (passos 3–6) e o que restou

3. ☒ Autenticar a CLI — feito com personal access token via `SUPABASE_ACCESS_TOKEN` (o fluxo de navegador não roda sem TTY)
4. ☒ `npm run db:types` — os três objetos apareceram; `gzbmhgnpoehormnidmagg` é o **gc-dev**
5. ☒ `npx tsc --noEmit` e `npm run lint` com os types novos — ambos limpos
6. ☒ `db:types` blindado contra truncamento e aceitando `$SUPABASE_PROJECT_ID`, com `db:types:dev` ao lado
7. ☒ Envolver 008 e 009 em `begin; ... commit;` — não afeta dev (já aplicadas), protege a aplicação futura em prod

Em espera — gc-prod pausado

8. [—] Rodar o script de verificação em gc-prod e diffar contra a saída de dev
9. [—] Backup + janela de manutenção antes de qualquer coisa em prod (a 004 é destrutiva: 13 colunas de `obras`)
10. [—] Aplicar o que faltar em prod **na ordem 004→010** e revalidar

Estrutural — feito

11. ☒ Removido o `as unknown as` das três páginas de listagem (`obras`, `fd`, `orcamentos`), trocado por um `as` só: mantém o narrowing dos campos NOT NULL e devolve ao `tsc` a comparação com o `Database` gerado. Verificado injetando uma coluna falsa no `select()` — o compilador acusa `column 'coluna_que_nao_existe' does not exist on 'obras_com_valores'`
12. ☒ Adotado `supabase/migrations` + CLI: as dez migrations viraram arquivos com timestamp (datas reais dos commits que as introduziram), `config.toml` criado e validado pela própria CLI, e o procedimento de adoção está em `supabase/README.md`

Pendente de execução por quem tem acesso: o `migration repair` que marca as dez como já aplicadas em gc-dev. Sem ele o histórico remoto fica vazio e um `db push` tentaria reaplicar a `revisao_schema` (ex-004). Comandos exatos no `supabase/README.md`.